

Visualización de Datos

Taller Práctico: Sensores, TTN y Firebase

Grupo de Investigación TIDOP

Henar y Miguel

Desarrolladores Web y Multiplataforma

Visualización de **datos**:

Somos desarrolladores en el grupo TIDOP.

Nos encargamos de la representación de datos:
la visualización en dispositivos móviles.

Nuestra meta es que conectéis vuestros sensores con
plataformas como TTN o Firebase y representéis la
información de forma clara y sencilla.



Nuestra **tarea** hoy



Conectar

Subir datos a plataformas en la nube como TTN y Firebase.



Visualizar

Crear una App móvil para Android o iOS sin complicaciones.



Interpretar

Representar la información de los sensores de forma legible.

Recapitulación

Punto de partida tras el Hardware

Del Sensor a la Pantalla

Partimos de un dispositivo configurado y funcionando.

El flujo que seguiremos es:

1. Hardware (Sensor) envía señal.
2. TTN recibe el paquete LoRaWAN.
3. Firebase almacena el histórico.
4. App Inventor muestra el dato.



Herramientas

Herramienta	Uso en el Taller
Móvil (Android/iOS)	Visualización final del dato
TTN (V3 Console)	Recepción de red LoRaWAN
Firebase	Persistencia y base de datos
App Inventor	Creación de la App por bloques

MIT App Inventor

Desarrollo Ágil y Didáctico

¿Por qué App Inventor?

Sencillez

Permite a alumnos y profesores crear proyectos funcionales en minutos usando bloques similares a Scratch.

Open Source

Es un proyecto de licencia libre, accesible para todo el mundo sin costes de licencia de desarrollo.

Alternativas

Kodular: Es un fork de AppInventor, más moderno, tiene otros componentes aunque parecidos. Tiene cierta monetización con algunas partes del proyecto.

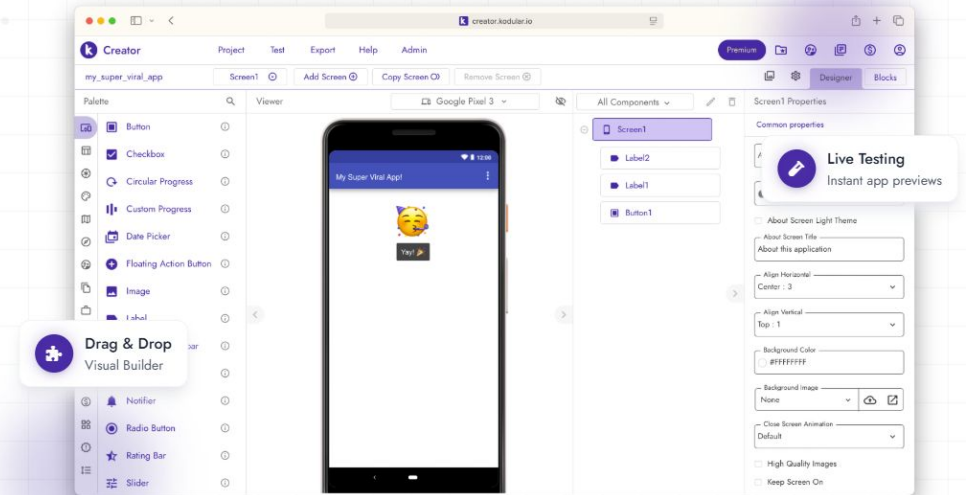
Kodular

Much more than a modern app creator without coding

Drag & Drop No Coding Publish Anywhere

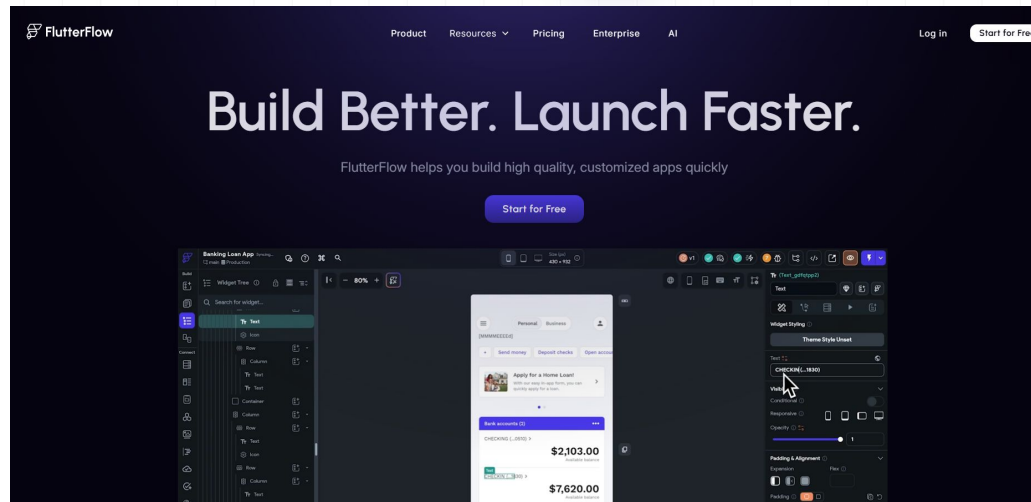
Create Apps

Learn More



POWERFUL EDITOR Build beautiful mobile apps with our intuitive interface

Flutterflow, un editor visual para uso de flutter con funciones basicas gratis y otras de pago



Darse de alta en App Inventor



Create Apps!

About

Learning

News & Events

Get Involved

Resources

Donate



Hacemos click aqui e iniciamos sesión con
google



+ New project



☐ Name

nuevo proyecto

Welcome to App Inventor!

Welcome to MIT App Inventor

We've Updated the default model used by the Chatbot Component.

[Read this blog post for more information.](#)

Got an iPhone or Android phone? Find out how to
[set up and connect an iOS or Android device.](#)

Continue

☐ Do Not Show Again

Create new App Inventor project

Project name:

Toolkit:

Default [?]

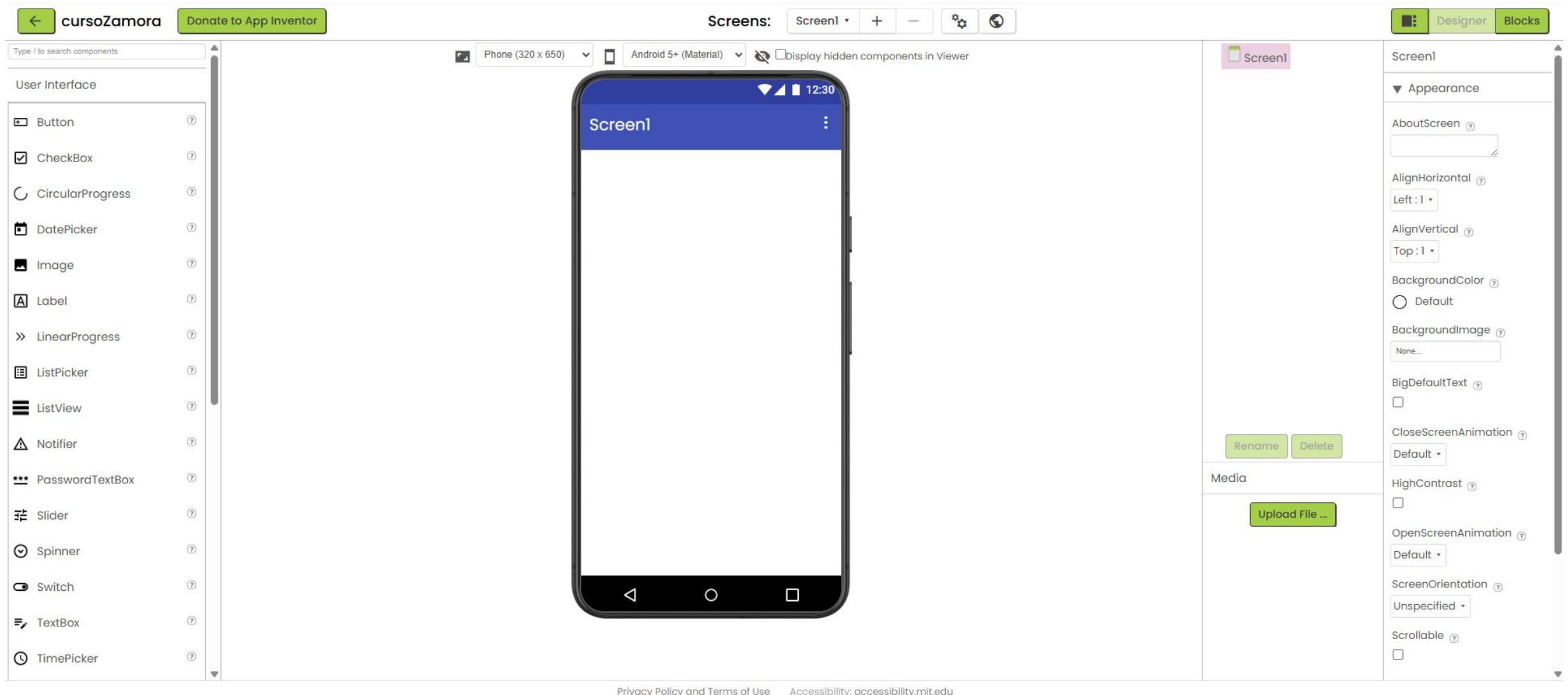
Theme:

Device Default [?]

Cancel

OK

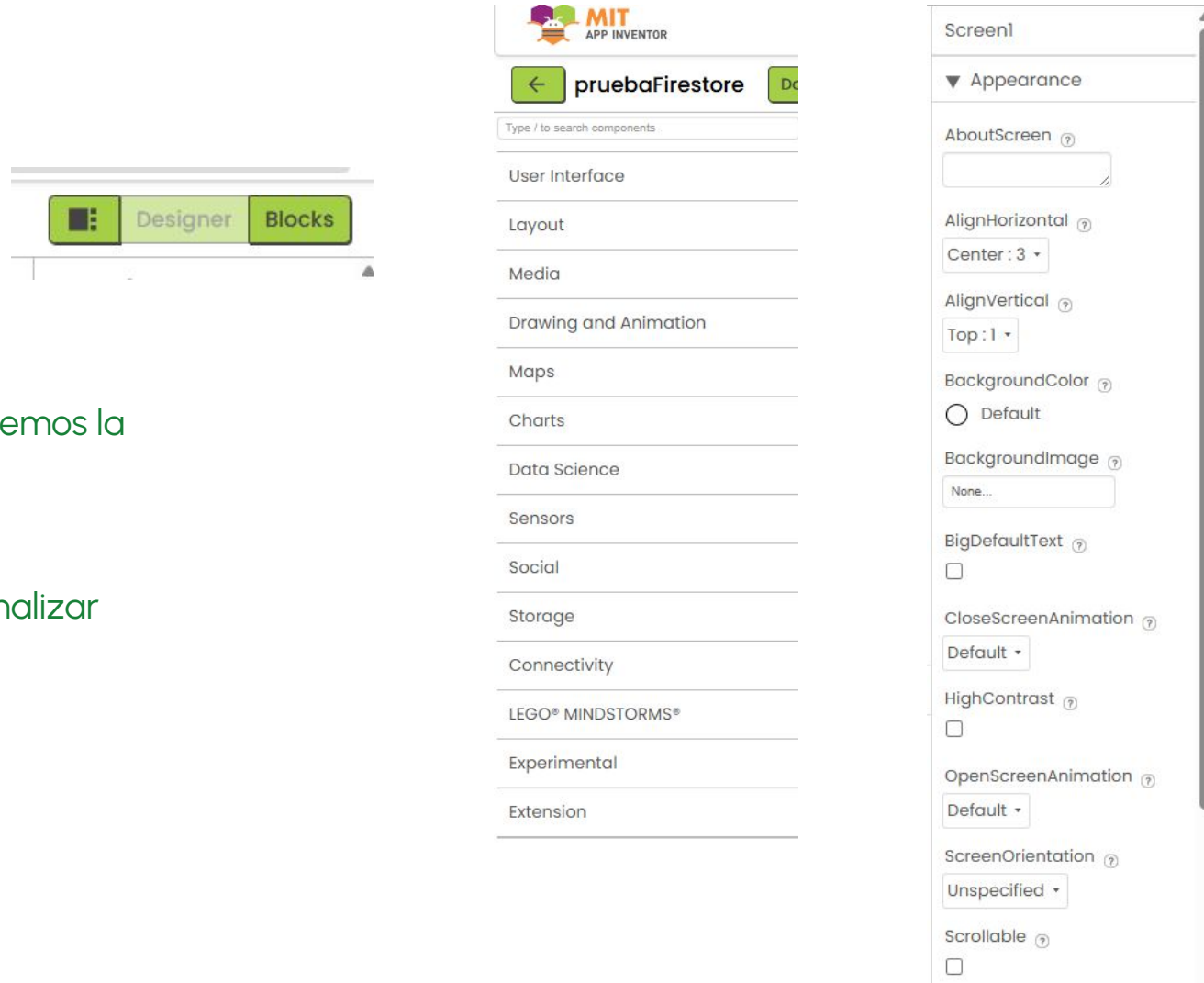
Vista general de App Inventor



Interfaz de Diseño

Aquí definimos el aspecto visual. A la izquierda tenemos la paleta de **Componentes** (botones, etiquetas).

A la derecha, el panel de **Propiedades** para personalizar colores, tamaños y alineaciones.



Lógica de Bloques



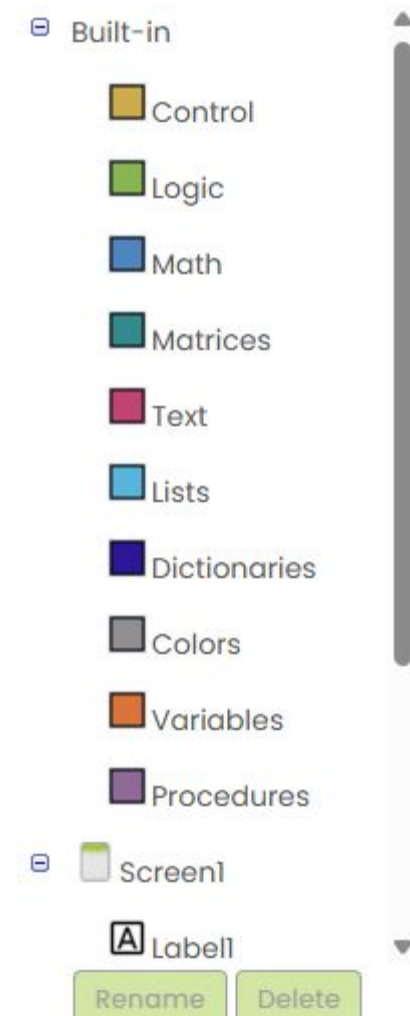
En la pestaña **Blocks** construimos la lógica de la aplicación.

Aunque no tiene buscador, los bloques están organizados por colores y categorías:

 **Control:** Bucles y saltos.

 **Text:** Cadenas y etiquetas.

 **Componentes:** Bloques específicos de tu UI.



Prueba y error: Debugging con IA

IA

¿Dudas con los bloques?

Usa herramientas como ChatGPT o Gemini. Haz una captura de tus bloques y dile qué quieres conseguir. ¡Te explicará el error visualmente!

Gemini



```
Plaintext
when Web1 GotText [url, responseCode, responseType, responseContent]
do
  // 1. Decodificamos el JSON completo de Firebase a un Diccionario
  initialize local [datosFirebase] to: [call Web1.JsonTextDecodeWithDictionary
                                         jsonText: [get responseContent]
                                         dictionary: [get responseContent]]
  in
    // 2. Preparamos la lista visual vacía
    initialize local [formattedList] to: [create empty list]
    in
      // 3. Extraemos solo los valores (ignoramos las claves r
      for each [registro] in list: [get dictionary values]
        dictionary: [get datosF
          do
            // 4. Montamos la tarjeta. ¡Ojo a las rutas sin '
            [add items to list]
            list: [get formattedList]
            item: [join]
              "Radon Actual: "
              [get value at key path]
              dictionary: [get registro]
              path: [make a list]
                "uplink_message"
                "decoded_payload"
                "radon_now"
```

Warnings y Logs

A diferencia de Android Studio, no tenemos Logcat.
Si algo falla, el móvil mostrará un aviso.

⚠ **Warnings:** Esquina inferior izquierda.

👤 **Logs:** Debes crearlos tú mismo en etiquetas de texto para depurar.

Práctica: Hola Mundo

- 1 Arrastra un **Button** y un **Label** al visor.
- 2 En Bloques, busca: when Button1.Click.
- 3 Encaja dentro: set Label1.Text to.
- 4 Añade una pieza de texto con: "¡Hola Mundo!".



AI Companion

Instala la App en tu Store (iOS/Android).

Menú Connect > AI Companion y escanea el QR.

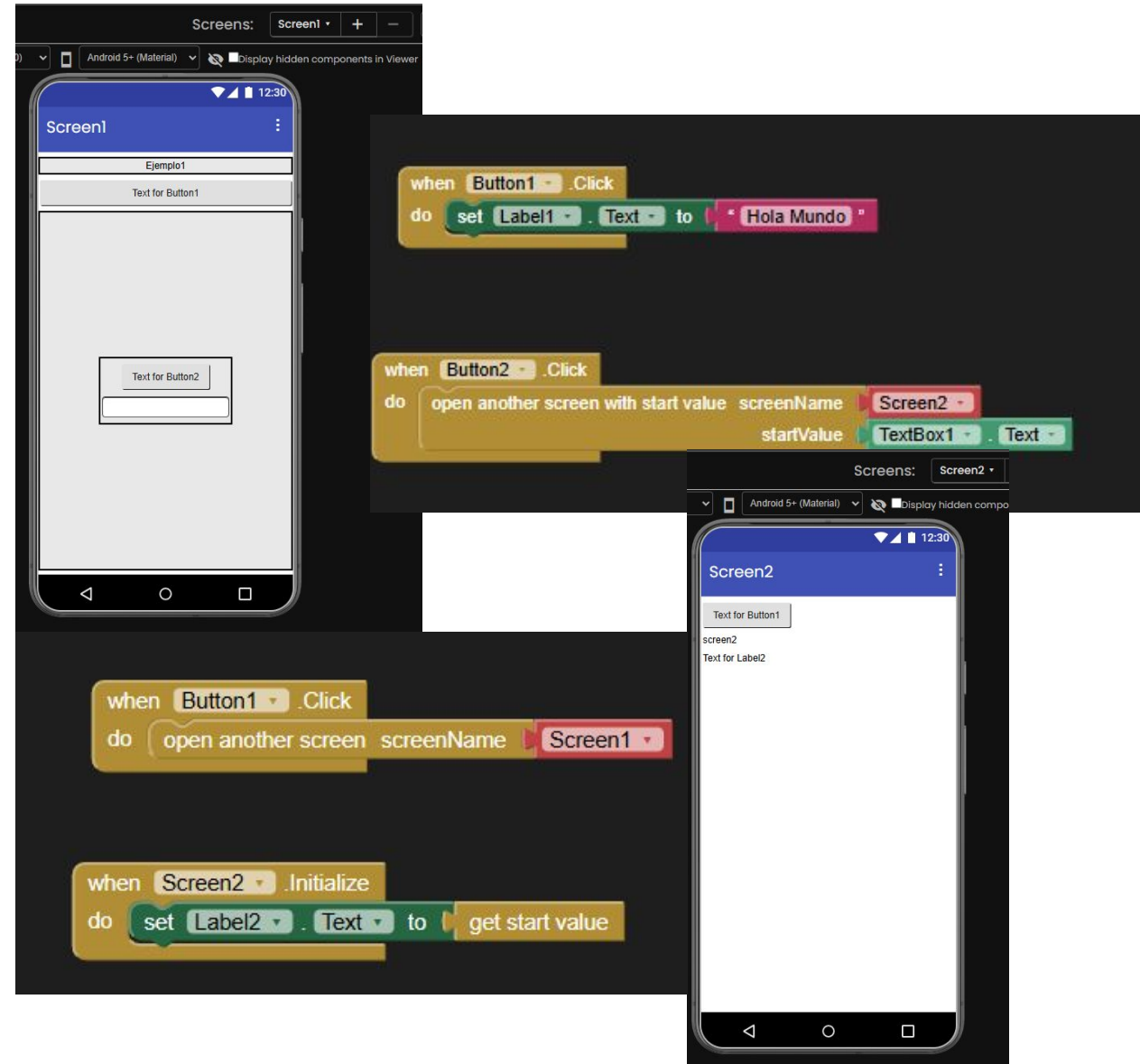
Páginas y Navegación

Multiscreen

Usa el bloque open another screen bajo la categoría Control. Puedes pasar parámetros (IDs) a la nueva ventana.

Truco

A veces es más eficiente ocultar y mostrar *Layouts* enteros en una misma pantalla que gestionar múltiples Screens.



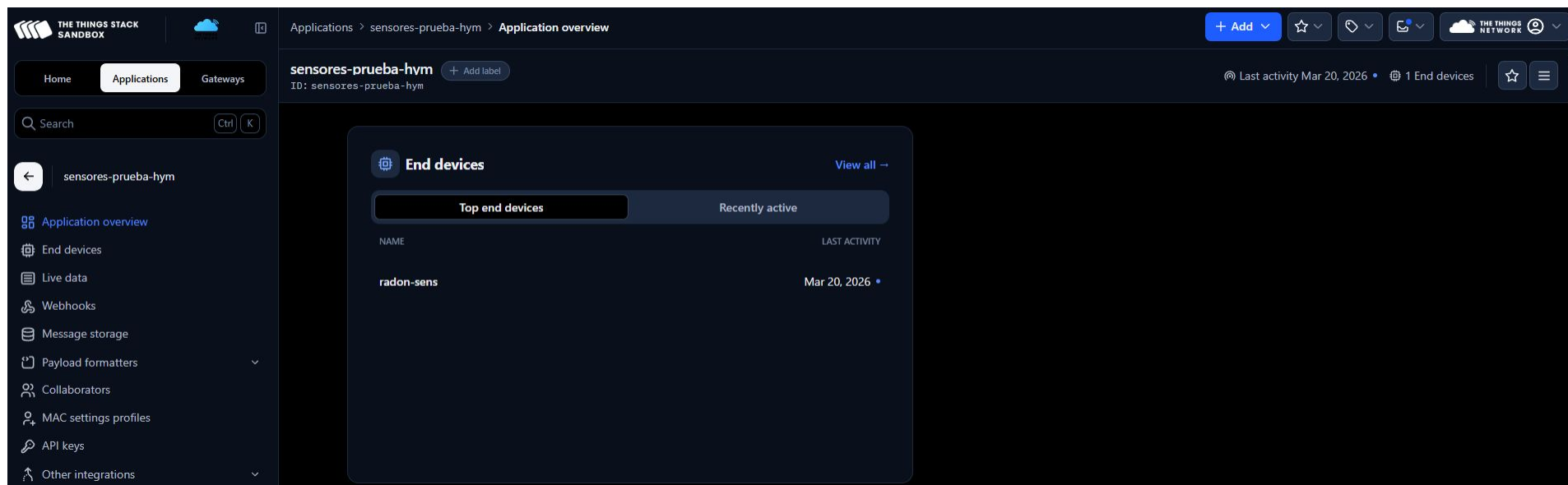
The Things Network

LoRaWAN para Internet de las Cosas

¿Qué es TTN?

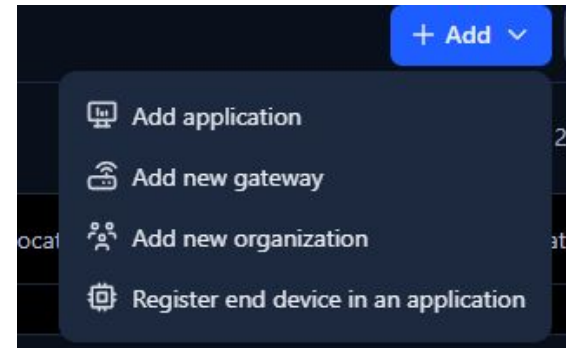
TTN es una red LoRaWAN europea que nos permite registrar dispositivos de forma gratuita.

Actúa como la "antena" que escucha a nuestros sensores y nos entrega los datos en un panel de control accesible.



Alta de Sensores

-  1. Crea una **Application** en la consola.
-  2. Añade un **End Device**.
-  3. Genera AppEUI y AppKey para flashear el sensor.
-  4. Activa el **Message Storage** para guardar el historial.



Provisioning information

JoinEUI ⓘ *
84 04 01 00 00 00 00 00 Reset
This end device can be registered on the network

DevEUI ⓘ *
| Generate 1/50 used

AppKey ⓘ *
.. Generate

End device ID ⓘ *
my-new-device

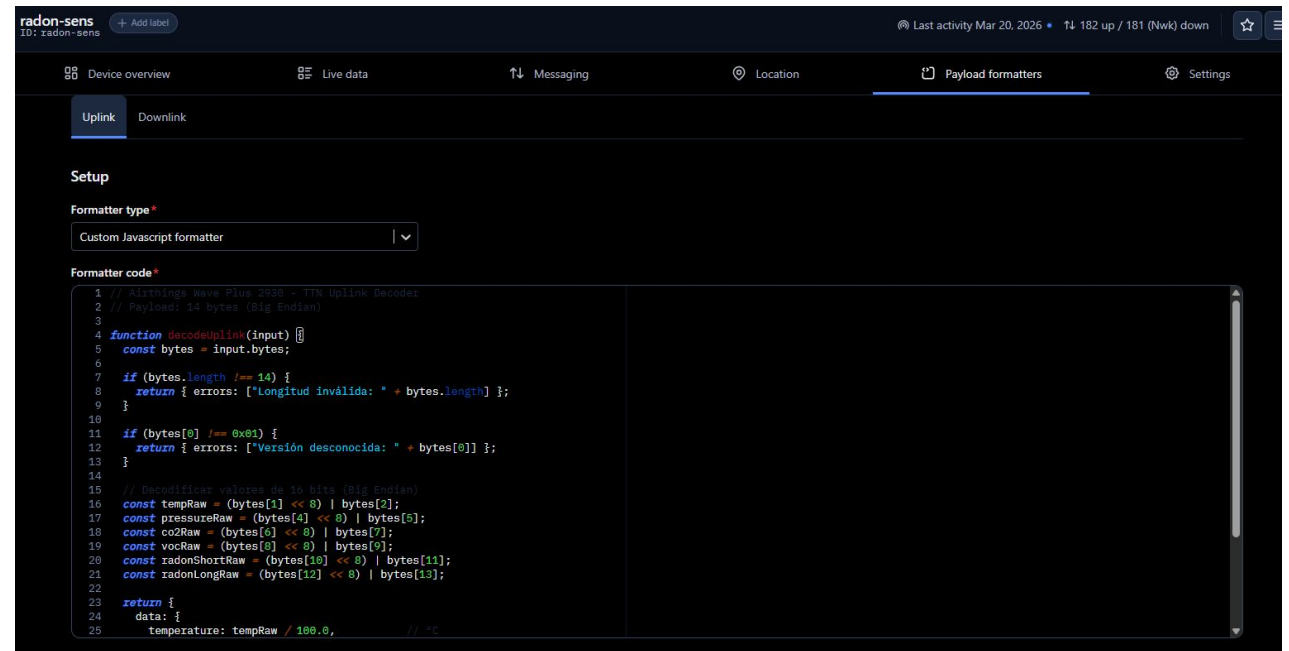
Device properties

Payload Formatters

Los sensores envían datos en **Hexadecimal** (ilegibles). El Formatter los traduce a JSON.

Es un "traductor" que convierte códigos extraños de un sensor en palabras y números que cualquier persona puede entender.

Si es un sensor propio (ESP32), debemos definir los patrones para separar temperatura, humedad o radón.

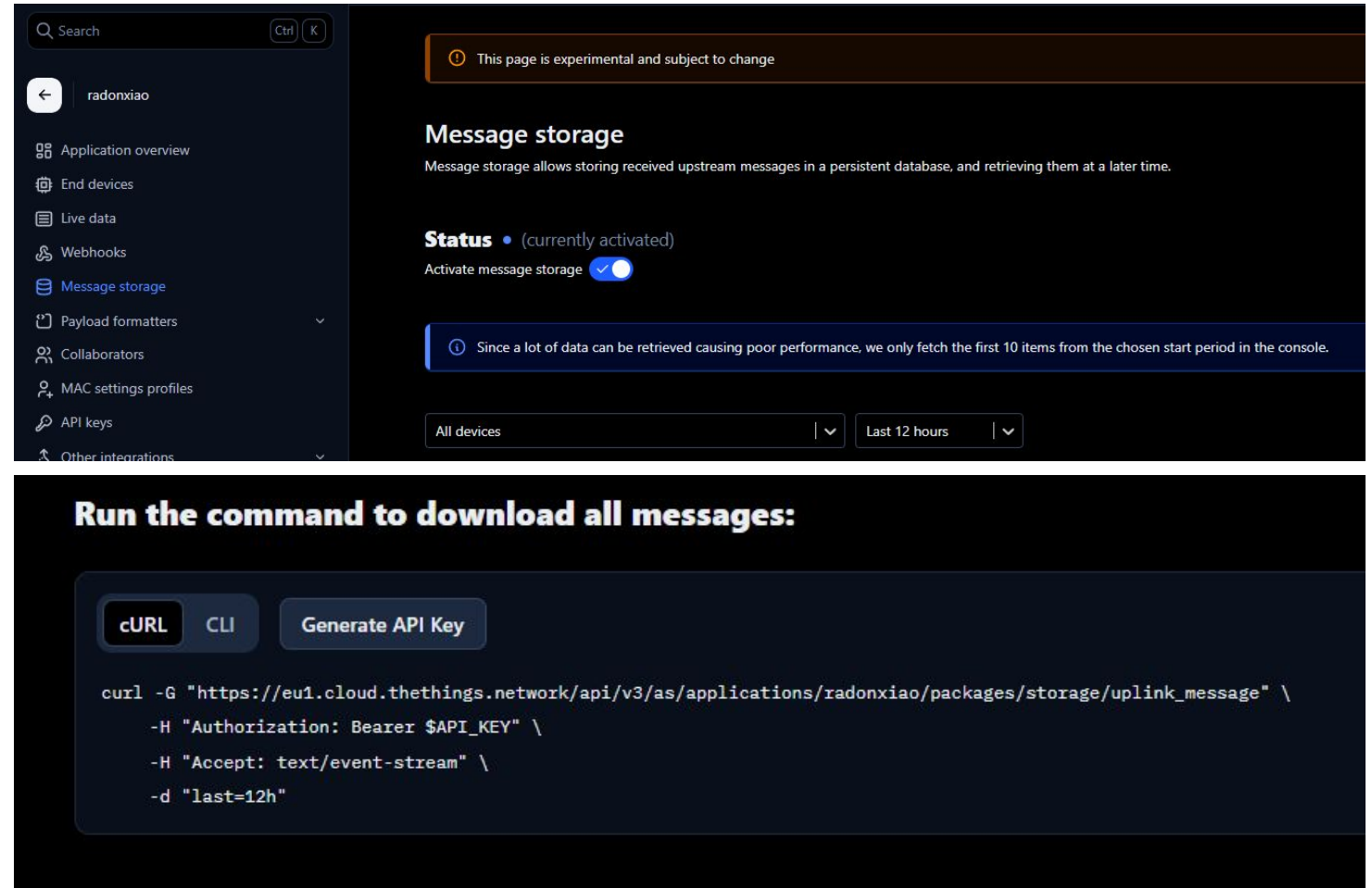


Message Storage y URL con filtrado por tiempo

Filtros de tiempo

Añade **?last=12h** a la URL de TTN para consumir solo las últimas 12 horas.

Necesitarás un **Bearer Token** para la autenticación.



The image shows two parts of the TTN interface. The top part is a screenshot of the 'Message storage' page for the application 'radonxiao'. It includes a sidebar with navigation links like 'Application overview', 'End devices', 'Live data', 'Webhooks', 'Message storage' (selected), 'Payload formatters', 'Collaborators', 'MAC settings profiles', 'API keys', and 'Other integrations'. The main content area shows a warning that the page is experimental, the 'Message storage' title, a description, and a status section indicating it is 'currently activated' with a toggle switch. Below this is a note about data retrieval limits and two dropdown menus for 'All devices' and 'Last 12 hours'. The bottom part of the image is a dark-themed box with the heading 'Run the command to download all messages:'. It contains two buttons, 'cURL' and 'CLI', and a 'Generate API Key' button. Below these is a terminal command to fetch messages using curl.

```
curl -G "https://eu1.cloud.thethings.network/api/v3/as/applications/radonxiao/packages/storage/uplink_message" \
-H "Authorization: Bearer $API_KEY" \
-H "Accept: text/event-stream" \
-d "last=12h"
```

Google Firebase

Persistencia y Webhooks

Persistencia de Datos

Histórico

TTN tiene un límite de almacenamiento.
Firebase nos permite guardar datos con mucho más margen para análisis a largo plazo.

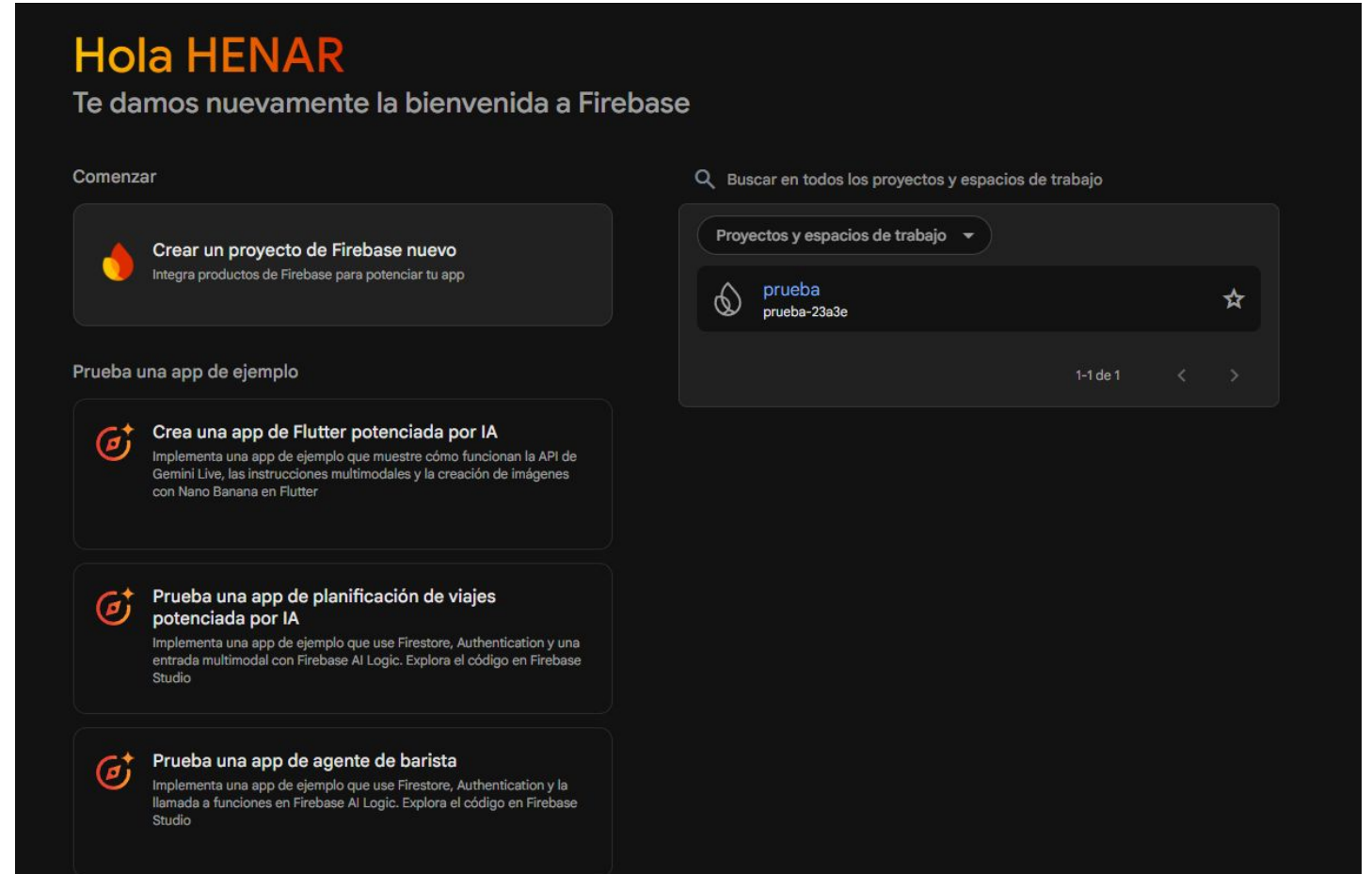
Sincronización (Webhook)

Cada vez que TTN recibe un dato, lo "empuja" a Firebase automáticamente mediante un Webhook.

¿Qué es Firebase?

Firebase es una plataforma de Google diseñada para facilitar el desarrollo de aplicaciones móviles y web.

Es un "todo en uno" que ahorra el trabajo pesado de configurar y mantener servidores. En lugar de construir toda la infraestructura desde cero, conectas tu app a Firebase y usas sus servicios listos para llevar.



¿Para qué sirve principalmente **Firestore**?

Bases de Datos en Tiempo Real: Permite que, si un usuario cambia algo (como un mensaje de chat), todos los demás lo vean al instante sin refrescar la pantalla.

Autenticación: Gestiona el inicio de sesión con correo electrónico de forma segura y en pocos minutos.

Hosting: Un lugar rápido y seguro para subir tu página web.

Almacenamiento (Storage): Para guardar fotos, videos o archivos de los usuarios.

Notificaciones: Envío de mensajes "push" a los celulares.

Nosotros lo utilizaremos como BBDD



Ejemplo de base de datos en **Firestore**

[Más funciones en Google Cloud](#)

Vista del panel Compilador de consultas

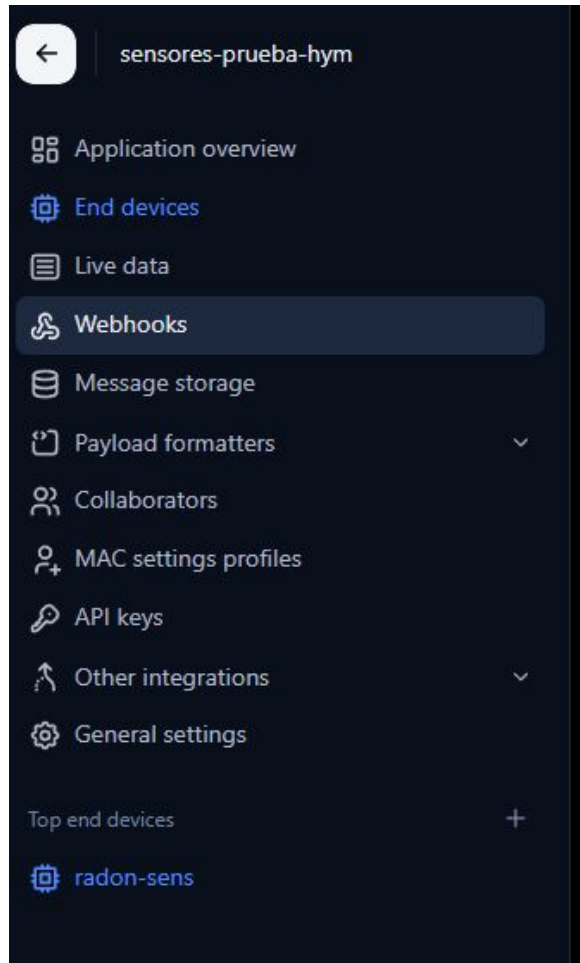
> usuarios > 4BC52otgDuilc...

(default)	usuarios	4BC52otgDuilcuhGkRni
+ Iniciar colección	+ Agregar documento	+ Iniciar colección
usuarios >	4BC52otgDuilcuhGkRni >	+ Agregar campo
	CbIJ0tcG5q1iLGsRst3g HYokK7LIwyp3h2JSFSEt atH2Bd4I78LJFnnox5jC	apellidos: "Herráez" nombre: "henar"

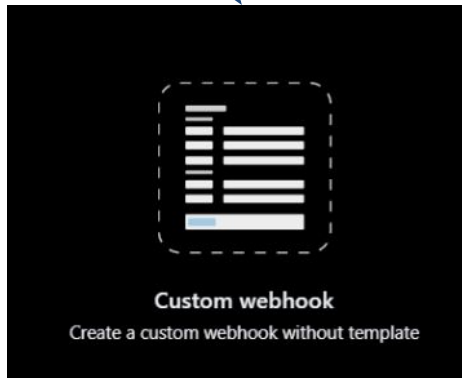
El Webhook de TTN



El Webhook de TTN



+ Add webhook



General settings

Webhook ID *

conexion-firebase

Webhook format *

JSON

Base URL *

https://cursozamora-60821-default-rtdb.firebaseio.com/

Downlink API key

Enabled event types

For each enabled event type an optional path can be defined which will be appended to the base URL

☒ **Uplink message** /historial_radon_sens

An uplink message is received by the application

Optimización del JSON

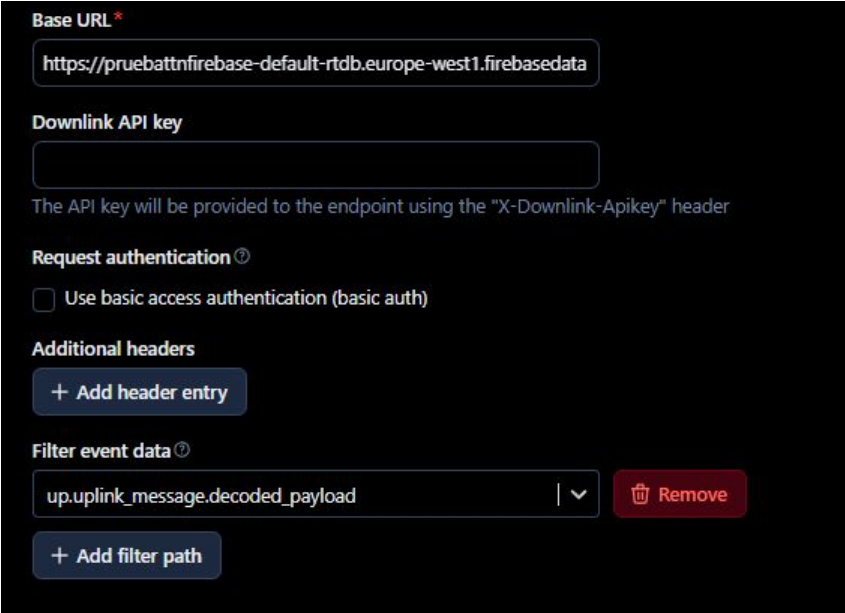
La plataforma TTN nos va a devolver siempre los valores "envueltos" en metadatos que representan cosas relacionadas con ellos tipo "intensidad de la señal, ruido, uid, gateway, etc". Para nuestro caso de uso estos son demasiados datos y necesitamos filtrarlos

Filtro en TTN:

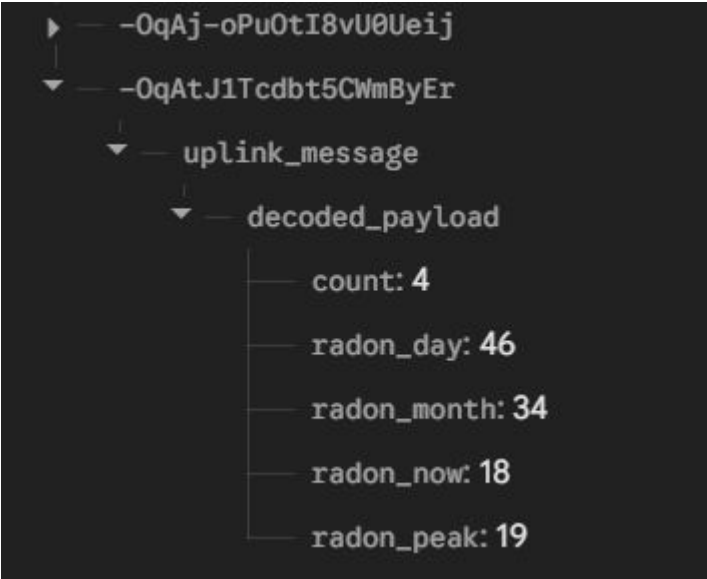
```
uplink_message.decoded_payload
```

Al aplicar este filtro, enviamos solo los datos limpios a Firebase, eliminando metadatos innecesarios.

Se pueden dejar más apartados tipo ids, timestamp, etc



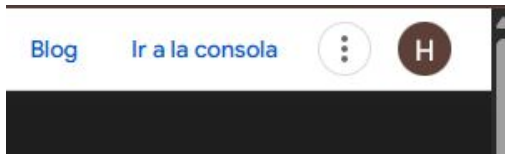
The screenshot shows the configuration for a Firebase endpoint in the TTN console. The 'Base URL' is set to 'https://pruebattnfirebase-default-rtdb.europe-west1.firebaseiodata'. The 'Downlink API key' field is empty, with a note stating 'The API key will be provided to the endpoint using the "X-Downlink-Apikey" header'. Under 'Request authentication', the 'Use basic access authentication (basic auth)' checkbox is unchecked. The 'Additional headers' section has a '+ Add header entry' button. The 'Filter event data' section shows a dropdown menu with 'uplink_message.decoded_payload' selected, and a '+ Add filter path' button. A 'Remove' button with a trash icon is also present.



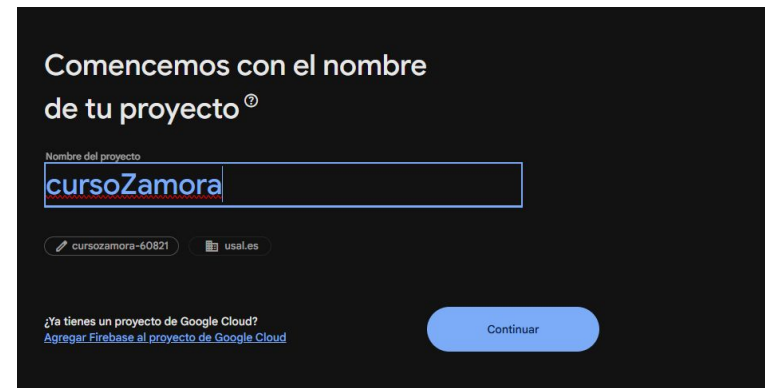
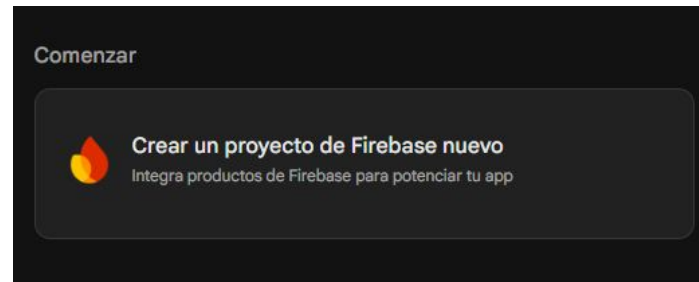
The screenshot shows a JSON tree view of the filtered data. The root node is expanded, showing a child node 'uplink_message', which is further expanded to show a child node 'decoded_payload'. The 'decoded_payload' node contains the following data: 'count: 4', 'radon_day: 46', 'radon_month: 34', 'radon_now: 18', and 'radon_peak: 19'.

Configuración de **Firebase**

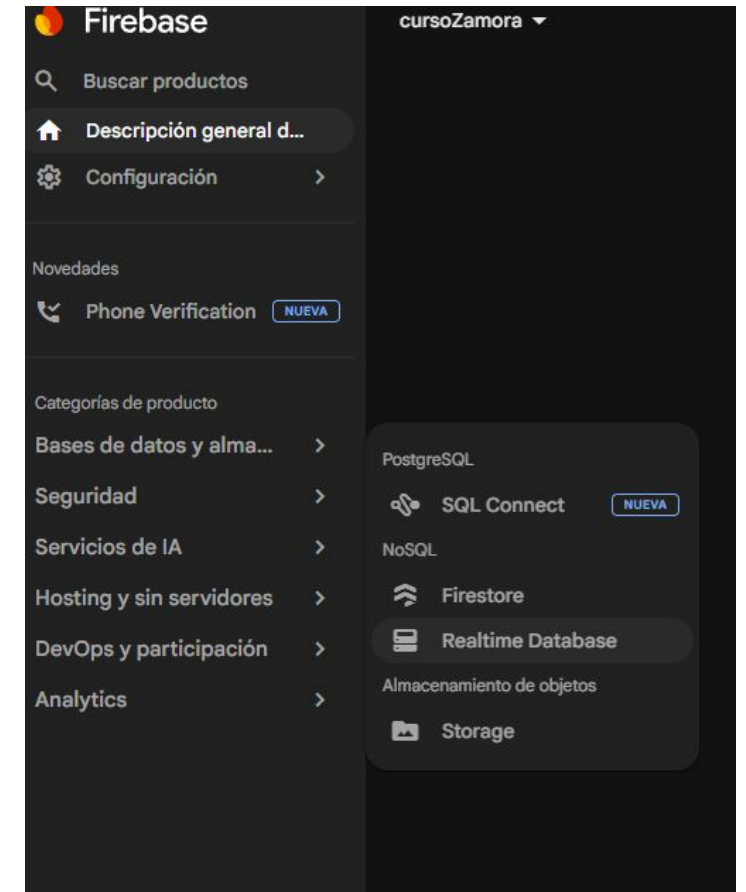
Una vez nos hemos registrado
debemos ir a la consola, situado
arriba a la izquierda



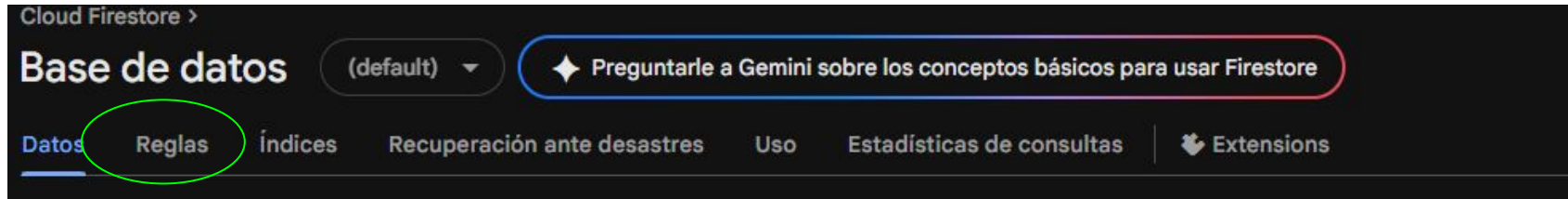
Creamos un proyecto nuevo



Seleccionamos RealTime Database



Configuración de Reglas



En este taller, abriremos la base de datos para facilitar la conexión:

```
1 {  
2   "rules": {  
3     ".read": true,  
4     ".write": true  
5   }  
6 }
```

* Nota: En producción, esto debe estar restringido. Quien tuviera la dirección podría escribir libremente.

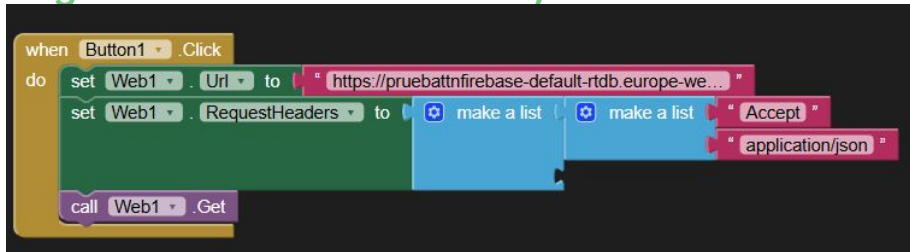
Listado

Listado de datos de Sensores en el Móvil

Pasos para mostrar una Lista

1. PETICIÓN (Web.Get)

Configuramos la URL de Firebase y lanzamos la llamada. Es el disparador que trae los datos de la nube al móvil.



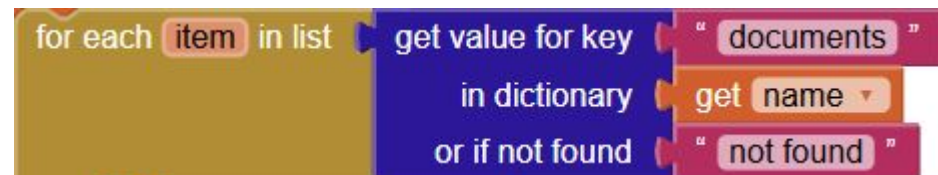
2. TRADUCCIÓN (JSON Decode)

Convertimos el responseContent (texto bruto) en un Diccionario. Este bloque es imprescindible para que App Inventor pueda "leer" las etiquetas del JSON.



3. FILTRADO (Key Path)

Accedemos a la raíz de los datos. Normalmente buscamos la clave principal (ej: documents o history) para obtener la lista de mensajes del sensor.



Pasos para mostrar una Lista

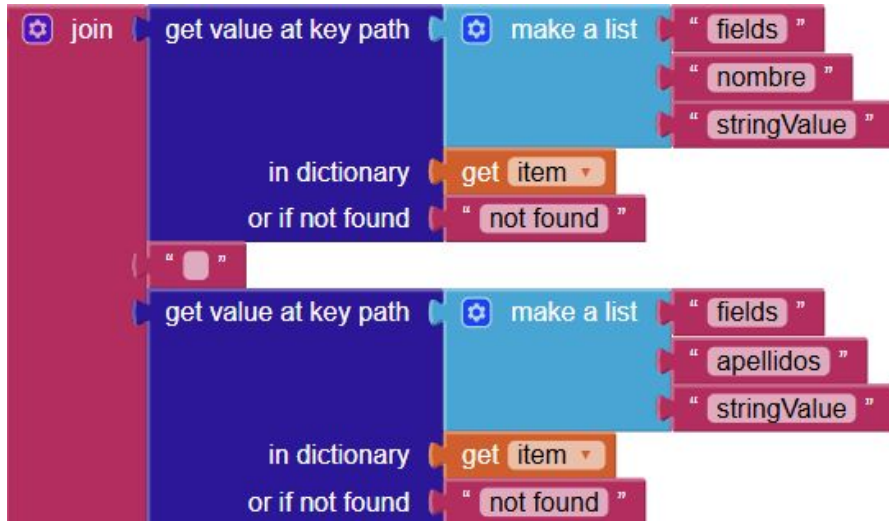
4. EXTRACCIÓN (Loop)

Ejecutamos un bucle for each item. Por cada registro que nos envía Firebase, extraemos el valor específico (ej: radon_now) y lo guardamos en una Variable Global.



5. MOSTRAR SÓLO LO QUE QUEREMOS.

Limpiamos el JSON para mostrar solo los datos que queremos, en este caso, nombre y apellidos de cada usuario



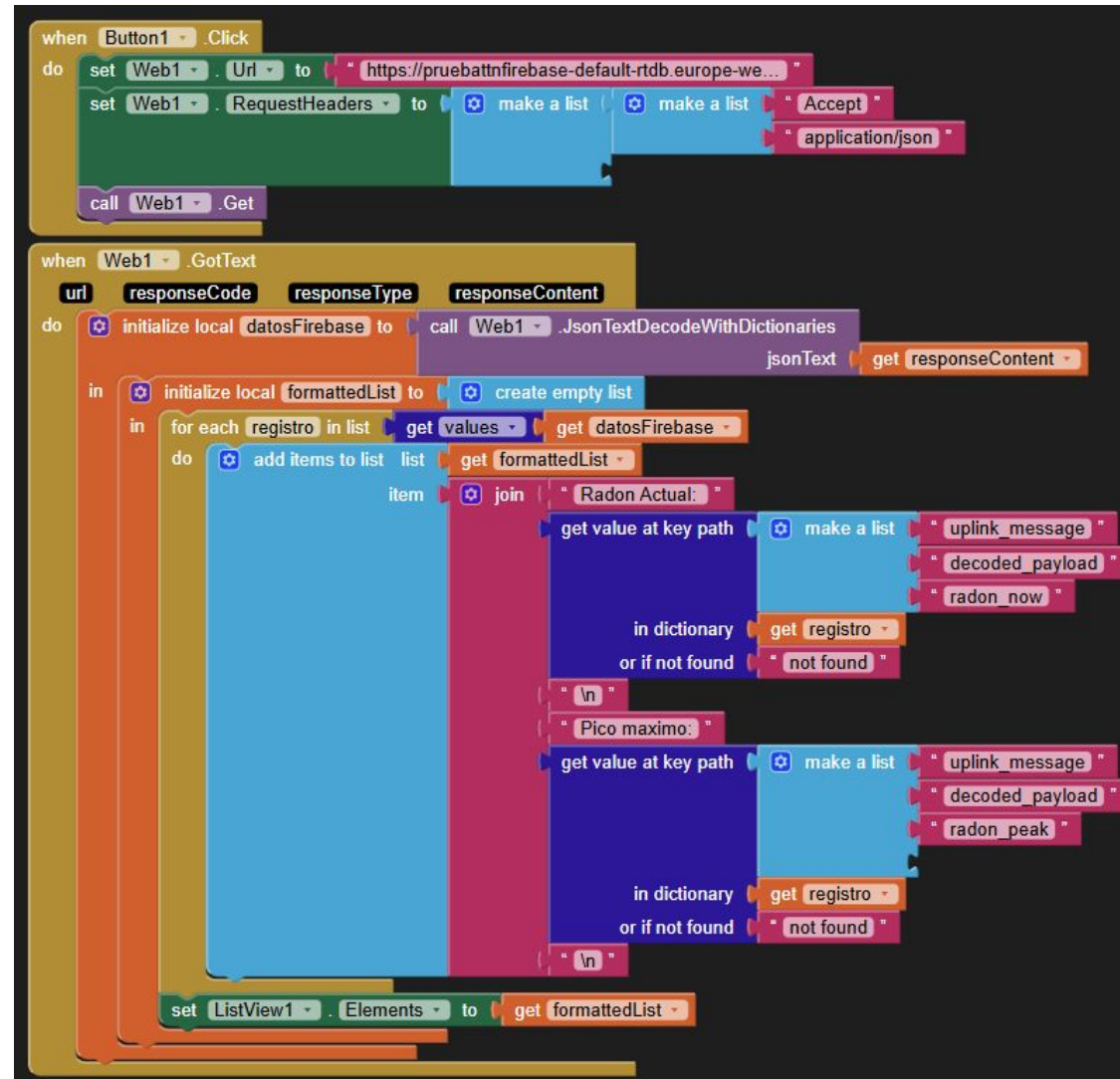
6. ASIGNAR A CADA ELEMENTO

Para cada elemento le asignamos un valor de la lista global



Pasos para mostrar una Lista

Resultado del bloque de la lista:



Gráficas simples

Gráfica de datos de Sensores en el Móvil

Pasos para mostrar un Gráfico

Para empezar a mostrar **gráfico** vamos a necesitar un par de cosas:

Lo hemo realizado con un botón para cargar pero se puede hacer que al cargar la página se cargue el gráfico por ejemplo.

¿Como agregar los elementos?

En el apartado izquierdo tenéis el apartado Charts que tiene 3 puntos:

Chart: El contenedor del gráfico

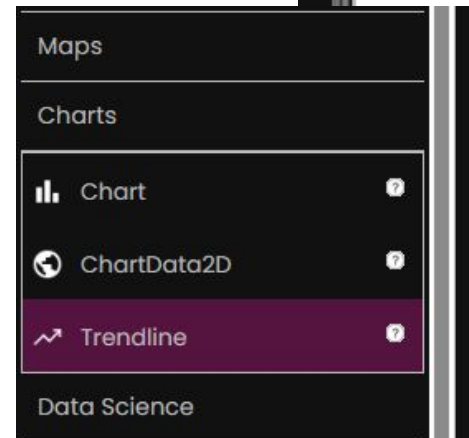
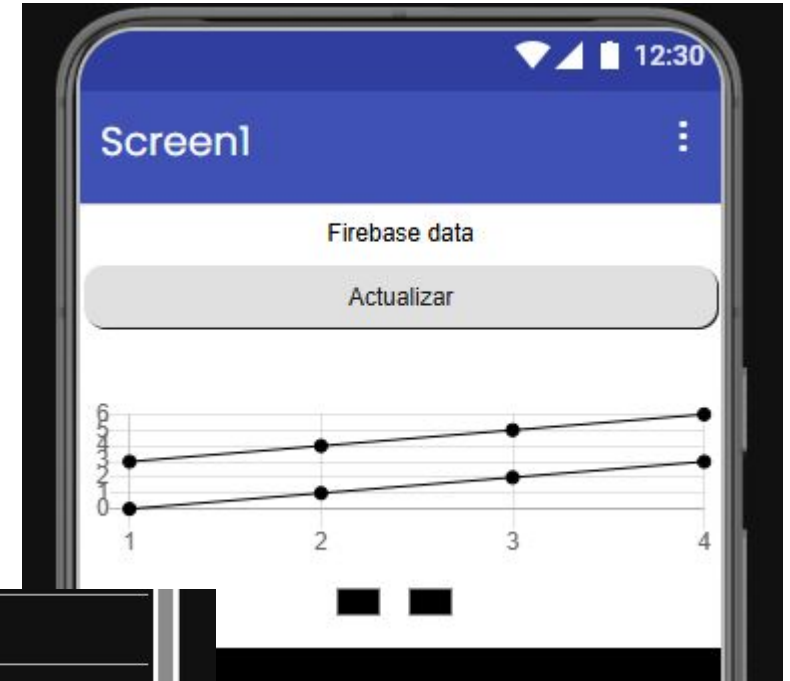
ChartData2D: El gráfico 2d a mostrar

Trendline: un componente que indica la tendencia de una serie de datos de una gráfica dada.

Para usar los gráficos tenemos, **primero, que arrastrar y soltar el componente Chart a la pantalla.**

Posteriormente tenemos que arrastrar el componente de ChartData2D dentro de ese Chart.

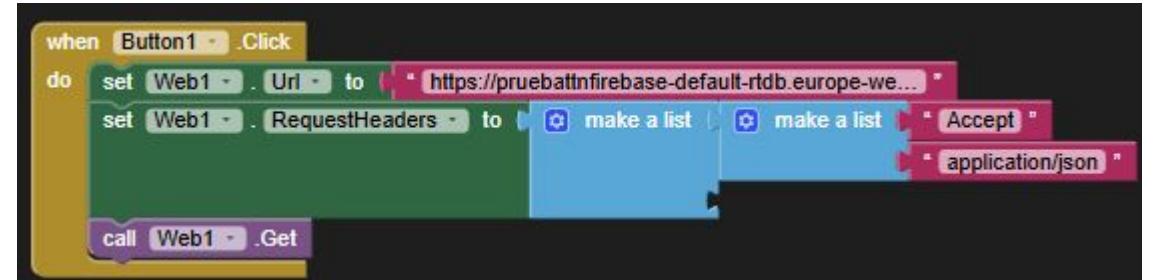
**Nota: En el caso de que queramos poner mas de un gráfico superpuesto tenemos que arrastrar más de un componente ChartData2D como veremos posteriormente.*



Pasos para mostrar un Gráfico

Objetivo: Lanzar la consulta HTTP a la base de datos (en este caso, Firebase Realtime Database) para descargar el histórico de registros.

- **Evento (Trigger):** `when Button1.Click`. La acción se desencadena manualmente por el usuario.
- **Configuración del Endpoint:**
 - Se define la URL de destino (`Web1.Url`). Apunta a un nodo específico de Firebase (`pruebattnfirebase...`).
 - Se establecen las cabeceras (`RequestHeaders`) indicando `Accept: application/json` para asegurar que el servidor devuelve una respuesta en formato JSON estándar.
- **Ejecución:** `call Web1.Get` ejecuta la llamada asíncrona.



Objetivo: Convertir la respuesta del servidor en estructuras de datos manejables y preparar los contenedores para la gráfica.

- **Evento de Recepción:** `when Web1.GotText`. Salta cuando Firebase devuelve el JSON con los datos.
- **Decodificación del JSON (Data Layer):**
 - `dictFirebase`: Convierte el texto JSON completo en un Diccionario. Firebase devuelve un objeto donde las claves son IDs alfanuméricos únicos y los valores son los datos en sí.
 - `listaDeLineas`: Se usa el bloque `get values` sobre el diccionario para ignorar esos IDs autogenerados y extraer únicamente una lista limpia con los objetos de datos (los *payloads*).
- **Inicialización de Lista de Coordinada:**
 - `data_actual`: Se inicializan como lista vacía. Aquí almacenaremos los puntos (x, y) de cada serie de la gráfica.
 - `x_counter`: Se inicializa en `1`. Servirá como eje X secuencial (tiempo o índice de lectura) para que la gráfica avance de izquierda a derecha.



*Nota: `data_actual` es mi dato de ejemplo pero ahí ponéis vosotros el vuestro

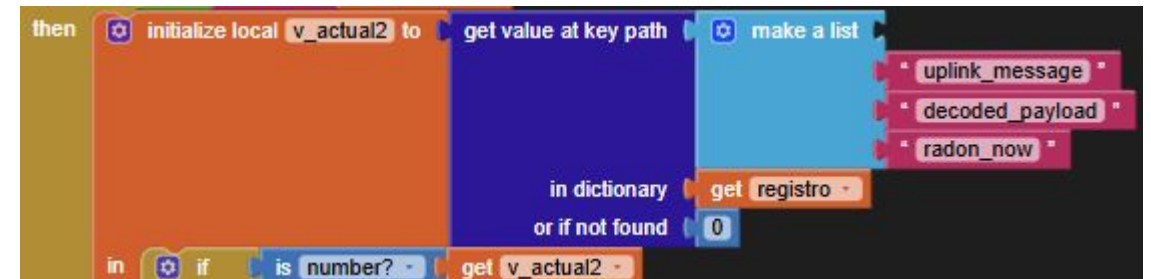
Pasos para mostrar un Gráfico

Objetivo: Recorrer el histórico, localizar los valores de radón y asegurar que sean válidos.

- **Iteración:** El bucle `for each registro` recorre cada objeto descargado.
- **Navegación por el JSON (Key Path):**
 - Se extraen las variables `v_actual2` (radón actual) y `v_pico2` (pico de radón) navegando por la ruta del JSON: `uplink_message -> decoded_payload -> radon_now / radon_peak`.
 - Se establece un valor por defecto de `0` (or `if not found 0`) para evitar errores nulos si el campo no existe.
- **Validación de Tipo (Safety Check):**
 - `if is number? ... and is number? ...`: Antes de pintar nada, el código verifica que ambos valores extraídos sean numéricos. Esto es una buena práctica para evitar que el componente de la gráfica lance una excepción y cierre la app si recibe un texto por error.



```
for each registro in list
do
  if not is empty
    get registro
```

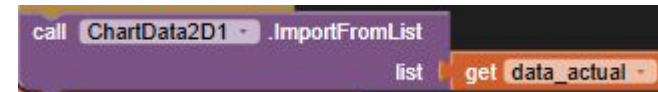


```
then
  initialize local v_actual2 to
  get value at key path
  make a list
    uplink_message
    decoded_payload
    radon_now
  in dictionary
  or if not found 0
  get registro
  in
  if is number?
    get v_actual2
```

Pasos para mostrar un Gráfico

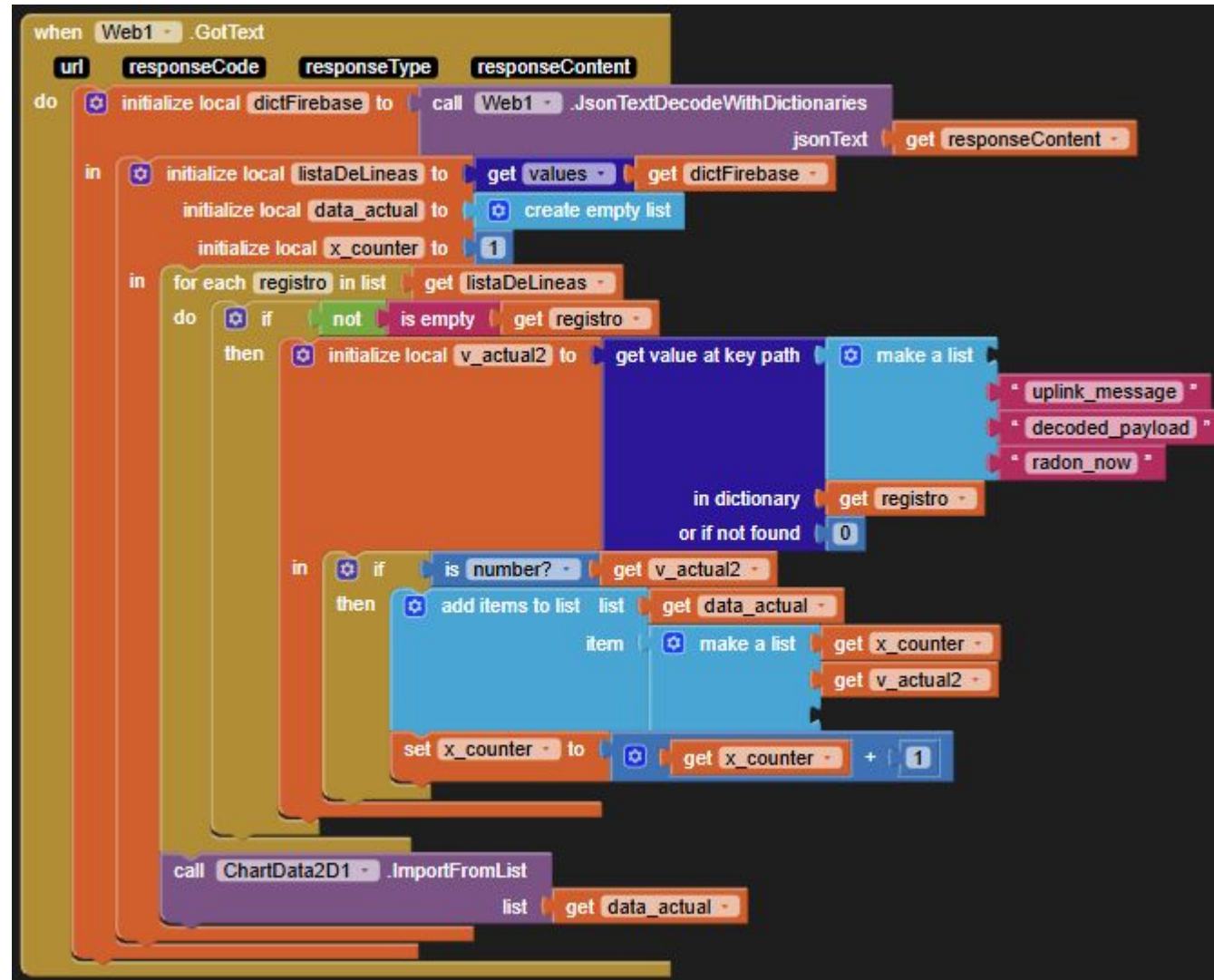
Objetivo: Formar los puntos del gráfico y volcarlos en la interfaz.

- **Creación de Puntos (X, Y):**
 - Si los datos son válidos, se crean listas de dos elementos [X, Y]: [x_counter, v_actual2] y [x_counter, v_pico2].
 - Estas pequeñas listas (coordenadas) se añaden a nuestras listas maestras `data_actual` y `data_pico`.
- **Incremento del Eje X:**
 - `set x_counter to get x_counter + 1`. Se suma 1 al eje X para que el siguiente punto se dibuje más a la derecha.
- **Volcado a la Interfaz (View Layer):**
 - Una vez que el bucle termina y todas las coordenadas están procesadas, se llama a las funciones `ImportFromList` de los componentes `ChartData2D1` y `ChartData2D2`.
 - Esto inyecta las listas de coordenadas directamente en las dos series del gráfico, renderizando la gráfica doble en la pantalla.



Pasos para mostrar un Gráfico

Resultado del bloque del gráfico:



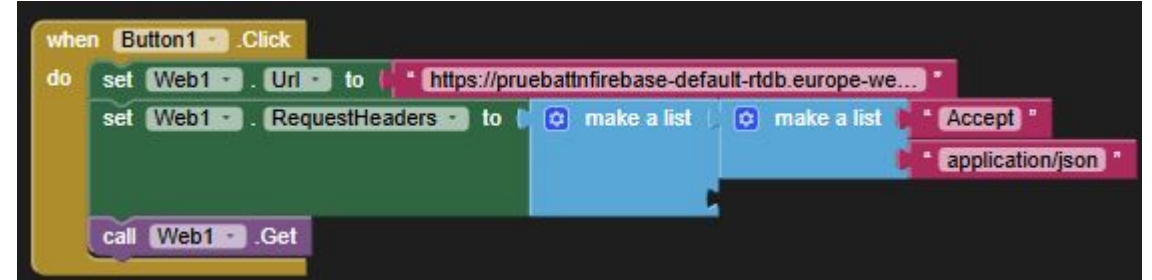
Gráficas dobles

Gráfica de datos de Sensores en el Móvil

Pasos para mostrar un Gráfico doble

Objetivo: Lanzar la consulta HTTP a la base de datos (en este caso, Firebase Realtime Database) para descargar el histórico de registros.

- **Evento (Trigger):** `when Button1.Click`. La acción se desencadena manualmente por el usuario.
- **Configuración del Endpoint:**
 - Se define la URL de destino (`Web1.Url`). Apunta a un nodo específico de Firebase (`pruebattnfirebase...`).
 - Se establecen las cabeceras (`RequestHeaders`) indicando `Accept: application/json` para asegurar que el servidor devuelve una respuesta en formato JSON estándar.
- **Ejecución:** `call Web1.Get` ejecuta la llamada asíncrona.



Objetivo: Convertir la respuesta del servidor en estructuras de datos manejables y preparar los contenedores para la gráfica.

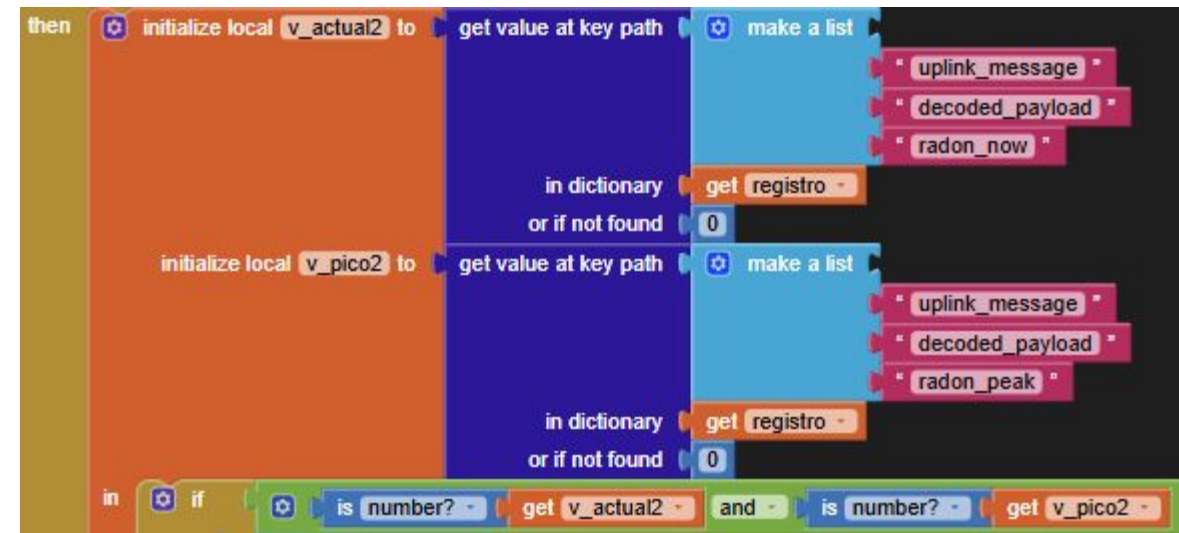
- **Evento de Recepción:** `when Web1.GotText`. Salta cuando Firebase devuelve el JSON con los datos.
- **Decodificación del JSON (Data Layer):**
 - `dictFirebase`: Convierte el texto JSON completo en un Diccionario. Firebase devuelve un objeto donde las claves son IDs alfanuméricos únicos y los valores son los datos en sí.
 - `listaDeLineas`: Se usa el bloque `get values` sobre el diccionario para ignorar esos IDs autogenerados y extraer únicamente una lista limpia con los objetos de datos (los *payloads*).
- **Inicialización de Listas de Coordenadas:**
 - `data_actual` y `data_pico`: Se inicializan como listas vacías. Aquí almacenaremos los puntos (x, y) de cada serie de la gráfica.
 - `x_counter`: Se inicializa en `1`. Servirá como eje X secuencial (tiempo o índice de lectura) para que la gráfica avance de izquierda a derecha.



Pasos para mostrar un Gráfico doble

Objetivo: Recorrer el histórico, localizar los valores de radón y asegurar que sean válidos.

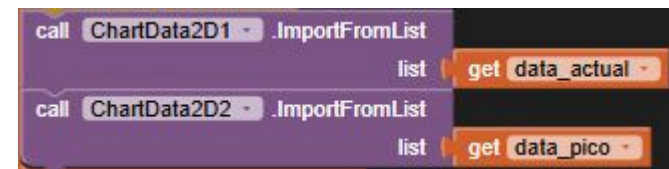
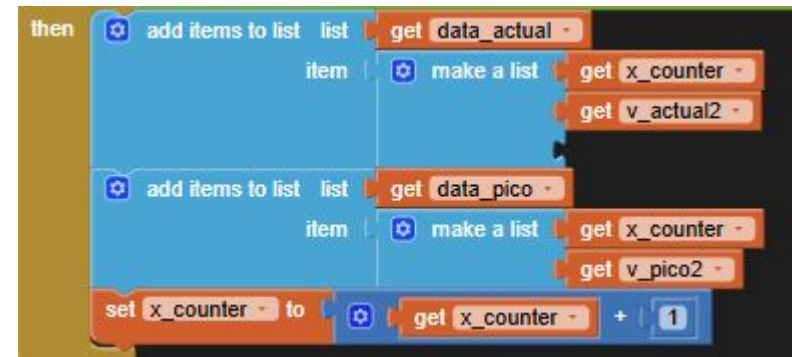
- **Iteración:** El bucle `for each registro` recorre cada objeto descargado.
- **Navegación por el JSON (Key Path):**
 - Se extraen las variables `v_actual2` (radón actual) y `v_pico2` (pico de radón) navegando por la ruta del JSON: `uplink_message` -> `decoded_payload` -> `radon_now` / `radon_peak`.
 - Se establece un valor por defecto de 0 (or `if not found 0`) para evitar errores nulos si el campo no existe.
- **Validación de Tipo (Safety Check):**
 - `if is number? ... and is number? ...`: Antes de pintar nada, el código verifica que ambos valores extraídos sean numéricos. Esto es una buena práctica para evitar que el componente de la gráfica lance una excepción y cierre la app si recibe un texto por error.



Pasos para mostrar un Gráfico doble

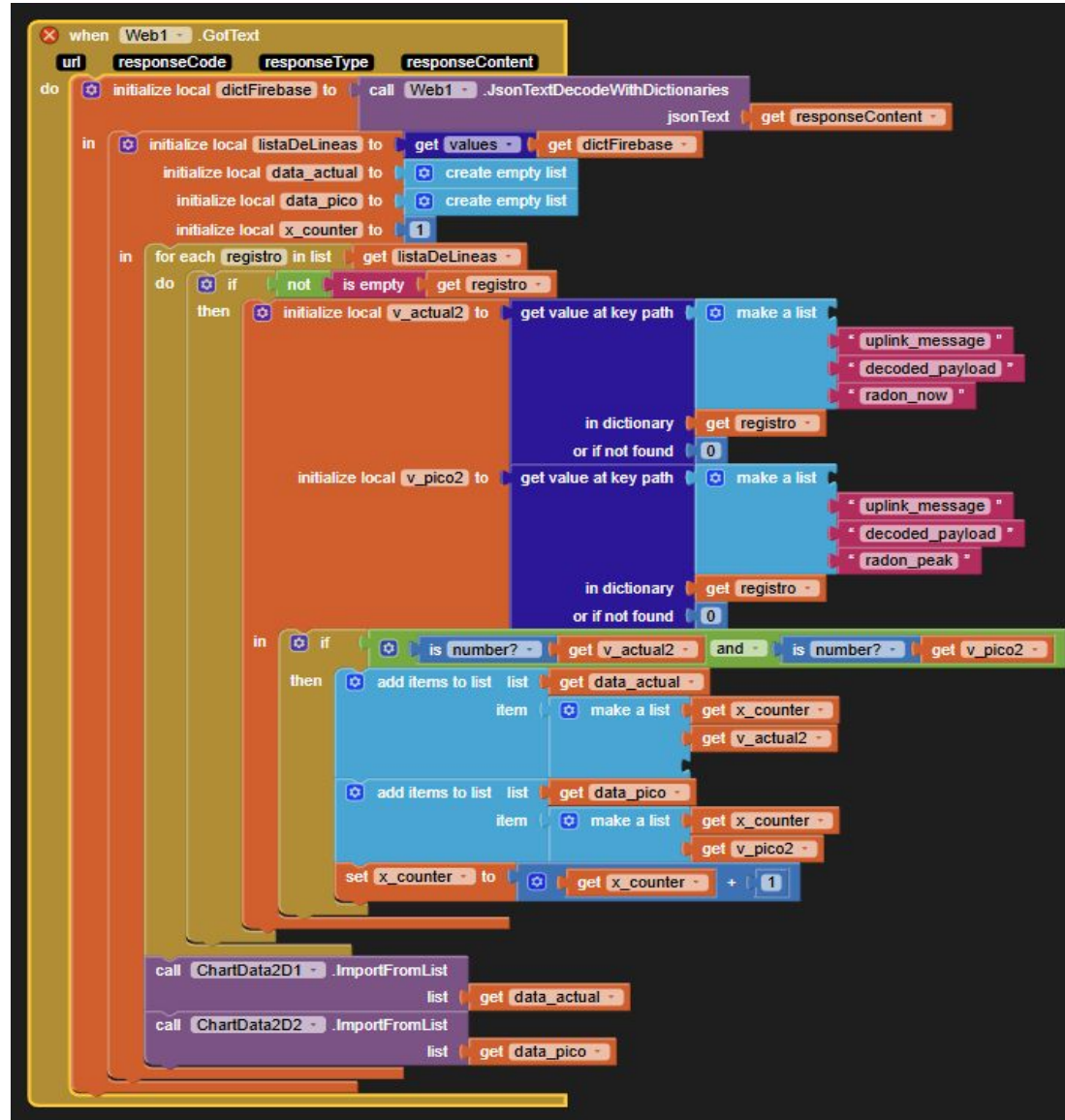
Objetivo: Formar los puntos del gráfico y volcarlos en la interfaz.

- **Creación de Puntos (X, Y):**
 - Si los datos son válidos, se crean listas de dos elementos [X, Y]: [x_counter, v_actua12] y [x_counter, v_pico2].
 - Estas pequeñas listas (coordenadas) se añaden a nuestras listas maestras data_actua1 y data_pico.
- **Incremento del Eje X:**
 - `set x_counter to get x_counter + 1`. Se suma 1 al eje X para que el siguiente punto se dibuje más a la derecha.
- **Volcado a la Interfaz (View Layer):**
 - Una vez que el bucle termina y todas las coordenadas están procesadas, se llama a las funciones `ImportFromList` de los componentes `ChartData2D1` y `ChartData2D2`.
 - Esto inyecta las listas de coordenadas directamente en las dos series del gráfico, renderizando la gráfica doble en la pantalla.



Pasos para mostrar un Gráfico

Resultado del bloque del gráfico:



Ej Aplicación Nativa

App MeCeloT Android + Android Studio

Checklist del Taller

1. TTN

Activar Storage y Webhook.

2. Firebase

Configurar reglas y limpiar base.

3. App Inventor

Conectar bloques y probar UI.

¿Dudas?

Henar y Miguel | TIDOP

¡Gracias por vuestra atención!